

LEVEL 12 — Reshaping Arrays

NumPy reshaping and dimension-manipulation methods — examples, output, and explanation line by line.

Function	Purpose
<code>reshape()</code>	Changes the shape without changing the data.
<code>resize()</code>	Changes the array size; it can change the total number of elements.
<code>flatten()</code>	Returns a copy as a 1-D array.
<code>ravel()</code>	Returns a flattened 1-D view when possible.
<code>transpose()</code>	Reverses axes by default or rearranges them with an axis order.
<code>.T</code>	Shortcut for <code>transpose()</code> for an array.
<code>swapaxes()</code>	Swaps two specified axes.
<code>moveaxis()</code>	Moves one or more axes to new positions.
<code>expand_dims()</code>	Adds a new axis of length 1.
<code>squeeze()</code>	Removes axes whose length is 1.

1. reshape()

Code:

```
import numpy as np

a = np.array([1, 2, 3, 4, 5, 6])
print(a)
b = a.reshape(2, 3)
print(b)
```

Output:

```
[1 2 3 4 5 6]
[[1 2 3]
 [4 5 6]]
```

Explanation: reshape(2, 3) changes six elements into 2 rows and 3 columns. The number of elements must stay the same.

2. resize()

Code:

```
a = np.array([1, 2, 3, 4, 5, 6])
a.resize(3, 2)
print(a)
```

Output:

```
[[1 2]
 [3 4]
 [5 6]]
```

Explanation: resize() changes the array in place. Unlike reshape(), the total size does not have to remain unchanged.

resize() — increasing size

Code:

```
a = np.array([1, 2, 3])
a.resize(2, 3)
print(a)
```

Output:

```
[[1 2 3]
 [0 0 0]]
```

Explanation: When the new size is larger, NumPy fills extra positions with zeros.

3. flatten()

Code:

```
a = np.array([[1, 2], [3, 4]])
b = a.flatten()
print(b)
```

Output:

```
[1 2 3 4]
```

Explanation: flatten() converts the array to one dimension and returns a copy.

4. ravel()

Code:

```
a = np.array([[1, 2], [3, 4]])
b = a.ravel()
print(b)
```

Output:

```
[1 2 3 4]
```

Explanation: `ravel()` also creates a 1-D array and tries to return a view when possible.

5. `transpose()`

Code:

```
a = np.array([[1, 2, 3],
[4, 5, 6]])
print(a.transpose())
```

Output:

```
[[1 4]
 [2 5]
 [3 6]]
```

Explanation: For a 2-D array, `transpose()` swaps rows and columns.

6. `.T`

Code:

```
a = np.array([[1, 2, 3],
[4, 5, 6]])
print(a.T)
```

Output:

```
[[1 4]
 [2 5]
 [3 6]]
```

Explanation: `.T` is a convenient shortcut for `transpose()` on an ndarray.

7. `swapaxes()`

Code:

```
a = np.array([[1, 2, 3],
[4, 5, 6]])
b = np.swapaxes(a, 0, 1)
print(b)
```

Output:

```
[[1 4]
 [2 5]
 [3 6]]
```

Explanation: Axis 0 and axis 1 are swapped.

8. `moveaxis()`

Code:

```
a = np.zeros((2, 3, 4))
print(a.shape)
b = np.moveaxis(a, 0, 2)
print(b.shape)
```

Output:

```
(2, 3, 4)
(3, 4, 2)
```

Explanation: `moveaxis()` moves axis 0 to position 2.

9. `expand_dims()`

Code:

```
a = np.array([1, 2, 3])
print(a.shape)
b = np.expand_dims(a, axis=0)
```

```
print(b)
print(b.shape)
```

Output:

```
(3,)
[[1 2 3]]
(1, 3)
```

Explanation: `expand_dims()` adds a new axis of length 1. `axis=0` adds it before the existing axis.

`expand_dims()` with `axis=1`

Code:

```
a = np.array([1, 2, 3])
b = np.expand_dims(a, axis=1)
print(b)
print(b.shape)
```

Output:

```
[[1]
 [2]
 [3]]
(3, 1)
```

Explanation: `axis=1` inserts the new length-1 axis after the original axis.

10. `squeeze()`

Code:

```
a = np.array([[1], [2], [3]])
print(a.shape)
b = np.squeeze(a)
print(b)
print(b.shape)
```

Output:

```
(1, 3, 1)
[1 2 3]
(3,)
```

Explanation: `squeeze()` removes all axes whose size is 1.

`squeeze()` with a chosen axis

Code:

```
a = np.array([[1], [2], [3]])
print(a.shape)
b = np.squeeze(a, axis=1)
print(b)
print(b.shape)
```

Output:

```
(3, 1)
[1 2 3]
(3,)
```

Explanation: `axis=1` removes only the specified length-1 axis.

Quick Revision

Operation	Main idea	Important point
reshape()	Change shape	Same number of elements
resize()	Change shape/size	Can add/remove elements; method changes array
flatten()	Make 1-D	Returns a copy
ravel()	Make 1-D	View when possible
transpose() / .T	Rearrange axes	2-D: rows ↔ columns
swapaxes()	Swap two axes	Specify two axes
moveaxis()	Move axes	Source axis → destination
expand_dims()	Add axis	New axis has size 1
squeeze()	Remove axis	Removes size-1 axes

Practice Questions

1. Create an array from 1 to 12 and reshape it into (3, 4).
2. Transpose the result and print its shape.
3. Flatten the transposed array.
4. Use `expand_dims(axis=0)` and `expand_dims(axis=1)` on a 1-D array.
5. Create an array with shape (1, 3, 1) and use `squeeze()`.
6. Create an array with shape (2, 3, 4) and move axis 0 to position 2.